ELSEVIER

Information Processing Letters 61 (1997) 113–120

Information Processing Letters

The incremental maintenance of a Depth-First-Search tree in directed acyclic graphs [1]

Paolo G. Franciosa [a,*], Giorgio Gambosi [b,2], Umberto Nanni [a,3]

[a] *Dipartimento di Informatica e Sistemistica, Università di Roma "La Sapienza", via Salaria 113, I-00198 Roma, Italy*

[b] *Dipartimento di Matematica, Università di Roma "Tor Vergata", via della Ricerca Scientifica, I-00133 Roma, Italy*

Received 1 March 1996
Communicated by S.G. Akl

Abstract

We propose an incremental algorithm to maintain a DFS-forest in a directed acyclic graph under a sequence of arc insertions in $O(nm)$ worst case total time, where n is the number of nodes and m is the number of arcs after the insertions. This compares favorably with the time required to recompute DFS from scratch by using Tarjan's $\Theta(n+m)$ algorithm any time a sequence of $\Omega(n)$ arc insertions must be handled. In particular, over a sequence of $\Theta(m)$ arc insertions our algorithm requires $O(n)$ amortized time per operation, and its worst case time is $O(n+m)$. Our algorithm relies on an original characterization of a DFS-forest in terms of a relaxed planar embedding of the graph. Besides the basic representation of the graphs in term of adjacency lists, $O(n)$ additional space is required. Although the problem of the dynamic maintenance of a DFS-tree was pointed out about one decade ago, this paper provides the first solution to this problem for nontrivial classes of graphs. © 1997 Elsevier Science B.V.

Keywords: Analysis of algorithms; Design of algorithms; Depth-first search

1. Introduction

Depth-First Search (DFS) is a very basic technique to explore graphs, which is used in several applications and represents a building block for the solution of many graph problems, such as connectivity testing, acyclicity testing, topological sorting, isomorphism of planar graphs, and others (see, e.g. [12,5,1,2]). A DFS of a graph can be efficiently performed in linear time [12], that is in time $O(n+m)$ for a graph $G = (N, A)$, where $n = |N|$, and $m = |A|$.

An efficient *dynamic* algorithm for DFS would provide a valuable tool to deal with several graph problems. In general, a dynamic algorithm for a problem P maintains the solution under a sequence of "small" changes in the input data without recomputing the solution from scratch. A number of graph problems have been given an efficient dynamic solution. Some problems seem to be much harder to be dealt with in a dynamic setting. As observed by Reif [10,11], DFS is one of the problems that, in the worst case, seem

* Corresponding author. Email: pgf@dis.uniroma1.it.

[1] Work supported by the EEC project "ESPRIT Long Term Research" No.20244 (ALCOM-IT), and by the Italian Project "Algoritmi, Modelli di Calcolo e Strutture Informative", Ministero dell'Università e della Ricerca Scientifica e Tecnologica (MURST). A preliminary version of this paper appeared in [4].

[2] Email: nanni@dis.uniroma1.it.

[3] Email: gambosi@mat.utovrm.it.

0020-0190/97/$17.00 © 1997 Elsevier Science B.V. All rights reserved.
PII S0020-0190(96)00202-5

to require a complete re-computation of the solution even if a single element of the input is modified. Reif puts DFS in a class of problems which are reducible to each other in such a way that a dynamic solution for any of these problems provides a dynamic solution for any other problem in the class. This class includes depth-first numbering, acceptance by linear time Turing Machines, path systems [3], boolean circuit evaluation [7], unit resolution [6]. Though, no solution is currently known for incremental DFS in nontrivial cases.

In this paper we provide a characterization of DFS-forests for directed graphs in terms of existence of a relaxed version of planar embedding of the graph itself. Then we consider the *incremental* DFS problem for the class of *directed acyclic graphs* (*dag*'s). We propose an algorithm for explicitly maintaining a DFS-forest under a sequence of arc insertions that do not introduce cycles in $O(mn)$ total time in the worst case, for a graph with n nodes and m arcs. This means $O(n)$ amortized time per arc insertion in a sequence of $\Theta(m)$ insertions. The bounds achieved compare favorably with the repeated application of Tarjan's $O(n+m)$ algorithm required to compute DFS from scratch: for long sequences of arc insertions on dense graphs the improvement is from $O(n^2)$ worst case to $O(n)$ amortized time per operation. Our space requirements, besides the standard $O(n+m)$ representation of the graph by using adjacency lists, consist in the representation of a suitable permutation of the nodes.

The proposed algorithm exploits the approach introduced in [9], where the authors obtain the same amortized time bound for the incremental maintenance of a topological sort of a *dag*.

This paper is organized as follows: after some basic definitions given in Section 3, an original characterization of DFS-forests for general directed graphs is introduced in Section 3, which is exploited in Section 4 to provide an incremental algorithm for *dag*'s.

2. Basic definitions

In the following we assume the standard graph theoretical terminology as in [1]. A *directed graph* $G = (N, A)$ (also referred to as a *digraph*) consists of a finite set N of *nodes* and a finite set of ordered pairs $A \subseteq N \times N$ of *arcs*, with $|N| = n$ and $|A| = m$. A *non-empty path* from v_1 to v_k is a sequence $\langle v_1, v_2\rangle, \langle v_2, v_3\rangle, \ldots, \langle v_{k-1}, v_k\rangle$ such that each $\langle v_h, v_{h+1}\rangle$ is an arc in A. An *empty path* connects any node v to itself. A *cycle* is a non-empty path from a node to itself. A digraph with no cycles is called a *directed acyclic graph* (in short *dag*).

Performing a visit of a digraph $G = (N, A)$, it is possible to determine a *spanning forest* $F = (N, A_F)$, with $A_F \subseteq A$ (see, e.g., [1,8]). A spanning forest F induces a partition of the arcs in A in four different classes:

- *tree arcs* (i.e., arcs in A_F) connect parent to children in F;
- *back arcs* connect descendants to ancestors;
- *forward arcs* connect ancestors to descendants other than children;
- *cross arcs* connect unrelated nodes in the tree.

A *DFS-forest* T of a directed graph is a spanning forest such that only cross arcs from right to left are allowed, assuming that the order of visit of the various components and within each component is from left to right.

In the following, for the sake of simplicity, each considered digraph $G = (N, A)$ is supposed to contain a node r (called the *root*) such that for each node in $x \in N - \{r\}$ there exists a path from r to x. This can be done without loss of generality; if a root is not available in a given graph G we might consider an extended digraph G' by adding a new dummy node r with an arc to each node in N: $G' = (N', A')$ with $N' = N \cup \{r\}$ $(r \notin N)$, and $A' = A \cup \{\langle r, v\rangle \mid v \in N\}$. Since there is a one-to-one correspondence between DFS-trees rooted at r on G' and DFS-forests on G, in the rest of the paper we only refer to DFS-trees, assuming that the visit is performed starting from a root r.

If we are given a graph G, it is possible to generate a rank function ϕ (that is, a bijection between N and the set of integers $\{1, \ldots, n\}$) by performing a depth-first search with a *postorder numbering* of the resulting tree. This numbering, which will be referred to as the *rank function* associated with a DFS-visit, can be recursively defined as follows (see, e.g., [1]). For any visited node x whose children in the tree are visited in the order $\langle v_1, v_2, \ldots, v_k\rangle$:

- number in postorder the subtrees rooted at v_1, $v_2, \ldots, v_k$;

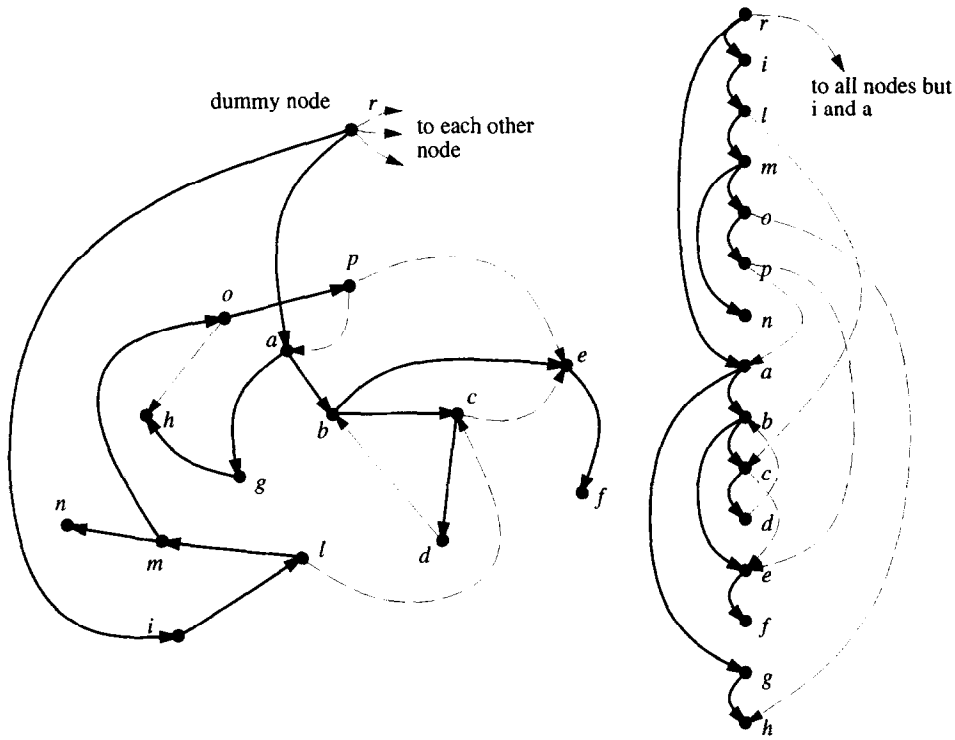

Fig. 1. A digraph and an associated one-sided planar spanning tree. DFS starts from node r.

- after all subtrees have been numbered, define $\phi(x)$ to be the smallest unused integer.

3. A characterization of DFS-trees

We characterize DFS-trees of a graph G by means of the existence of a suitable embedding of this graph in the plane.

Let $G = (N, A)$ be a digraph, $T = (N, A_T)$ be any spanning tree of G, and let us consider any permutation of N with a rank function ϕ. The *one-sided linear embedding* of (G, T) according to ϕ is obtained by positioning all nodes on a vertical line (with node u above node v if and only if $\phi(u) > \phi(v)$) and drawing arcs in T on the left of the vertical arrangement and arcs not in T on the right. Any arc $\langle u, v\rangle$ is said *downward* if $\phi(u) > \phi(v)$, otherwise it is said *upward*.

Definition 1. Let $G = (N, A)$ be a directed graph, and $T = (N, A_T)$ be any spanning tree of G. T is a *one-sided planar spanning tree* of G if there exists a permutation of nodes in N with rank function ϕ such that the one-sided linear embedding of (G, T) according to ϕ can be drawn on a plane with the following properties:

(1) all arcs in A_T are directed downwards;
(2) arcs in A_T do not intersect each other;
(3) for each upward arc $\langle u, v\rangle \in G$, there exists a tree-path $\langle v \equiv x_0, x_1\rangle, \langle x_1, x_2\rangle, \ldots, \langle x_k, x_{k+1} \equiv u\rangle$, with $\langle x_i, x_{i+1}\rangle \in A_T$, for each $i = 0, \ldots, k$.

An example of one-sided planar spanning tree is shown in Fig. 1.

The next theorem is just a different statement, in terms of one-sided planar spanning trees, of a well-known property of DFS-trees (see for example [1, Section 5.4]).

Theorem 2. *Let $G = (N, A)$ be a digraph, and $T = (N, A_T)$ be any DFS-tree of G. Then T is a one-sided planar spanning tree of G.*

Proof. In order to define a linear embedding of (G,T), let us consider the rank function ϕ generated by the postorder numbering of the given DFS-tree T.

It is obvious from the construction of the rank function that arcs in A_T can be drawn downward (from nodes with higher rank to nodes with lower rank) on the left halfplane without any intersection. On the other side, for each arc $\langle p,q\rangle \in A - A_T$ one of three cases is possible:

(a) p is a predecessor of q in T: in this case $\phi(p) > \phi(q)$, hence $\langle p,q\rangle$ is directed downward;

(b) q is a predecessor of p in T: in this case there is a tree-path from q to p in T. Then by construction $\phi(q) > \phi(p)$, and Case 3 in Definition 1 holds;

(c) p and q are not comparable in T: let $l = LCA_T(p,q)$ be the lowest common ancestor of nodes p and q in T, and let v_p (respectively, v_q) be the child of l which is a predecessor of p (respectively, q). Since $\langle p,q\rangle \in A - A_T$, the subtree rooted at v_q has been visited before the subtree rooted at v_p, thus $\phi(p) > \phi(q)$ and $\langle p,q\rangle$ is directed downward. □

The next theorem shows that *any* one-sided planar spanning tree is indeed a DFS-tree.

Theorem 3. *Let $G = (N,A)$ be a digraph, and $T = (N,A_T)$ be any one-sided planar spanning tree of G. Then T is a depth-first-search tree of G.*

Proof. We consider a visit of G that starts from the node of maximum rank (that is, from the root) and, for each entered node x, first traverses outgoing tree arcs in such a way that arc $\langle x,u\rangle$ is traversed before arc $\langle x,v\rangle$ if $\phi(u) < \phi(v)$, then traverses non tree arcs (in any order). We will show that this visit defines a depth-first-search for G.

Suppose now we are visiting node u, and consider any arc $\langle u,v\rangle \in A - A_T$. We show that node v has already been visited during the DFS visit. Two possible cases arise:

- $\langle u,v\rangle$ is directed downward, i.e. $\phi(u) > \phi(v)$:
  - if there exists a tree-path from u to v, then v has already been visited, since tree arcs outgoing from u have been traversed first;
  - if there is no tree-path from u to v, let $l = LCA_T(u,v)$; then by the visiting order assumed, since u lies above v, the whole subtree attached to l containing v has been visited before the subtree containing u;
- $\langle u,v\rangle$ is directed upward, i.e. $\phi(u) < \phi(v)$. By Definition 1, there exists a tree-path from v to u: this implies that v is visited before u in the DFS visit.

Hence all arcs in the right halfplane are either *forward*, *back* or *cross* arcs directed to the "left" (i.e., to nodes already visited), which implies that, since T is a spanning tree, it is actually a DFS-tree. □

Theorems 2 and 3 thus provide the following characterization of a DFS-tree:

Corollary 4. *Given a digraph G, a tree T is a DFS-tree of G if and only if T is a one-sided planar spanning tree of G.*

In the case of *dag*'s, the next corollary provides a simpler characterization of one-sided planar spanning trees that will be exploited in the following section.

Corollary 5. *Given a directed acyclic graph $G = (N,A)$, a spanning tree $T = (N,A_T)$ of G is a DFS-tree of G if and only if it is possible to draw G with all the nodes on a vertical line, with all arcs in T on the left and all arcs in $A - A_T$ on the right, so that the following properties hold:*

- *all arcs in A are directed downward;*
- *arcs in A_T do not intersect each other.*

4. Incremental maintenance of a DFS-tree in a *dag*

In this section we consider the problem of maintaining a DFS-tree of a *dag* G under a sequence of arc insertions; to this aim we will exploit the characterization provided in Corollary 5. The adopted strategy is described in the following. For any node $x \in N$, let $\phi(x)$ be the rank of node x in the postorder numbering of T. When a new arc $\langle u,v\rangle$ is inserted, if $\phi(u) > \phi(v)$ then no update is required to the current DFS-tree, since the conditions stated in Corollary 5 are fulfilled. In the case that $\phi(u) < \phi(v)$, a new DFS-tree for the graph $G' = (N, A \cup \{\langle u,v\rangle\})$ must be computed, together with a new permutation ϕ'. We will prove that it is possible to update the DFS-tree by considering only the following set of nodes:

$$I_G(u,v) = \{p \mid \phi(u) \leqslant \phi(p) \leqslant \phi(v)\}.$$

In Fig. 2 we show the procedures performing the required updates.

Procedure Insert carries out the insertion of a new arc $\langle u, v\rangle$, and if any update is required, the two following procedures are called. Procedure P-DFS, starting from arc $\langle u, v\rangle$, performs a partial DFS within the set $I_G(u, v)$ and determines the subset M of these nodes which are reachable from v. Such a DFS is performed by using the following pruning rules:

- if the partial DFS reaches node u, then the insertion of arc $\langle u, v\rangle$ introduces a cycle, and a message is reported;
- if a node z is entered such that $\phi(z) < \phi(u)$, then node z is not visited.

As an output of the partial DFS, a new permutation for nodes in the set M is computed (and stored by setting a relative rank function $\overline{\phi}$), and (possibly new) parents for these nodes are chosen. Procedure Update_rank computes the new value of the rank function $\phi'(p)$ for each node $p \in I_G(u, v)$, by using the relative ordering $\overline{\phi}$ for nodes in M, and shifting ahead nodes in $I_G(u, v) - M$ without changing their relative positions. Note that a new rank function ϕ' is introduced only in order to highlight the changes occurred, and therefore it is assumed that $\phi'(q) \equiv \phi(q)$ for each $q \notin I_G(u, v)$.

Fig. 3 shows a *dag* and its one-sided planar spanning tree before and after inserting an arc.

Theorem 6. *Let $G = (N, A)$ be a dag, and $T = (N, A_T)$ be any DFS-tree of G with postorder numbering ϕ. Then, after the insertion of arc $\langle u, v\rangle$, procedure* Insert *determines whether the new arc introduces a cycle, and, if the graph is still acyclic, correctly updates T and ϕ.*

Proof. Let $T' = (N, A_{T'})$ and ϕ' be respectively the tree and the rank function returned by procedure Insert after the insertion in A of arc $\langle u, v\rangle$, and let M be the set of nodes visited by procedure P-DFS while inserting arc $\langle u, v\rangle$.

Note that in T' each node has exactly one incoming arc (except for the root): this is enforced by line 6 of procedure P-DFS (in Fig. 2), where a (possibly new) parent is computed for each node $z \in M$. These new tree arcs are determined during a partial depth-first search that generates the subtree of T' with root v. On the other side, the existing tree-path from r to u and the new arc $\langle u, v\rangle \in A'_T$ provide the initial part of all the paths from r to nodes in M. Thus, since all nodes in $N - \{r\}$ have a single parent and are reachable from the root, T' is a spanning tree for G'.

In the following we show that, according to rank ϕ':

(a) all arcs in A' are directed downward;
(b) arcs in $A_{T'}$ do not intersect each other.

```
   Procedure Insert(⟨u,v⟩ : arc);
1.  begin{all nodes are supposed to be unmarked}
2.    Update(G,⟨u,v⟩);
        {the new arc is inserted in the graph}
3.    if φ(u) < φ(v)
4.    then begin
5.      m := 0;
6.      P-DFS(⟨u,v⟩); {a partial DFS is carried out}
7.      Update_rank(φ,φ̄,φ',m);
8.    end;
9.  end;

   Procedure P-DFS(⟨w,z⟩ : arc);
1.  begin
2.    if z = u then HALT(''a cycle has been detected'');
3.    if z is unmarked and φ(z) > φ(u)
4.    then begin{otherwise the search is pruned}
5.      mark(z);
6.      A_T := A_T − {⟨parent(z),z⟩} ∪ {⟨w,z⟩};
7.      for each arc ⟨z,y⟩ ∈ A do P-DFS(⟨z,y⟩);
8.      m := m + 1;
9.      φ̄(z) := m;
10.   end
11. end;

   Procedure Update_rank(φ,φ̄,φ',m);
1.  begin
2.    shift := m
3.    for i := φ(u) to φ(v) do
4.      begin
5.        let p_i be the node such that φ(p_i) = i;
6.        if p_i is marked
7.          then begin
8.            φ'(p_i) := φ(u) + φ̄(p_i) − 1;
9.            unmark(p_i);
10.         end;
11.         else begin
12.           φ'(p_i) := φ(u) + shift;
13.           shift := shift + 1;
14.         end;
15.     end;
16. end;
```

Fig. 2. Maintaining a DFS while inserting an arc $\langle u, v\rangle$.

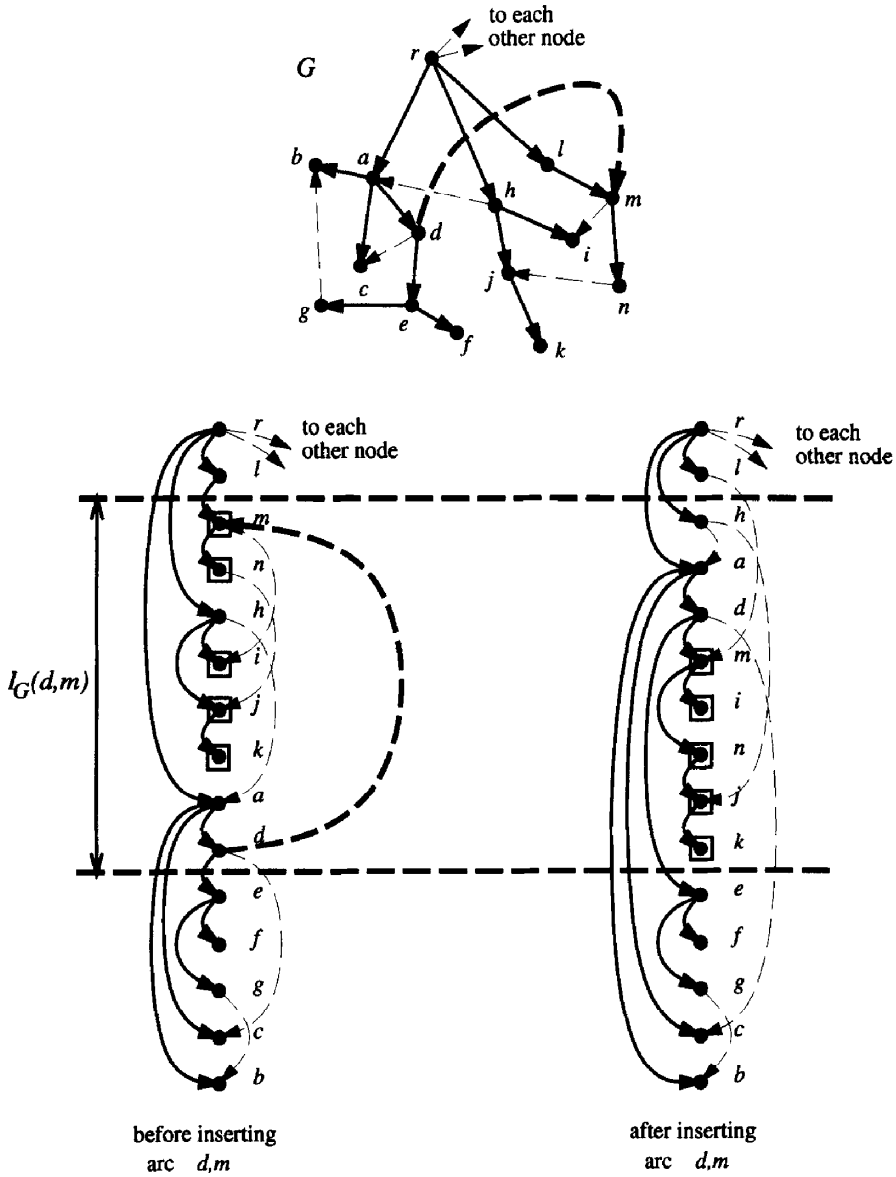

Fig. 3. A *dag* and its associated one-sided planar spanning tree before and after inserting an arc $\langle d, m\rangle$. Nodes in the set M (i.e., visited by procedure P-DFS) are marked by a small square.

Due to Corollary 5, these properties imply the theorem.

While performing the partial DFS, procedure P-DFS computes the relative rank function $\overline{\phi}$ for all nodes in the set M. Then procedure Update_rank scans all the nodes in the set $I_G(u,v)$ and, starting from rankings ϕ, $\overline{\phi}$ and from the quantity $m = |M|$, returns in time $O(|I_G(u,v)|)$ a new rank function ϕ' which is characterized in the following way:

- for $x \in N - I_G(u,v)$ we have that $\phi'(x) = \phi(x)$,
- for $x \in I_G(u,v)$:
  - if both x and y are in $I_G(u,v) - M$, then their relative order in ϕ' is unchanged with respect to ϕ;

- if $x \in I_G(u,v) - M$ and $y \in M$ then $\phi'(x) > \phi'(y)$, i.e., x is above y in the new linear arrangement (note, in particular, that $u \in I_G(u,v) - M$ and $v \in M$);
- if both x and y are in M, then their relative order in ϕ' is defined by the partial DFS performed by procedure P-DFS.

Due to these properties all arcs in the new arrangement are directed downward, since only nodes in the set $I_G(u,v)$ have changed their positions, and, on the other side, there exists no arc $\langle x,y\rangle$ with $x \in M$ and $y \in I_G(u,v) - M$ since y would have been visited during the partial DFS from v.

Concerning property (b) above, we have that, rearranging the nodes according to the new ordering ϕ', no intersection has been introduced in T' between tree arcs with both endpoints in $N - M$, since all these nodes have not changed their relative order and their parent with respect to T (see Fig. 3). Also tree arcs with both endpoints in M (that in the new arrangement are placed in consecutive positions) can be drawn with no intersection, since a new partial DFS of nodes in M generates a one-sided planar spanning tree (see Theorem 2). Furthermore, no tree arc leaves the set M to reach a node in $N - M$ and, finally, the only tree arc from a node $N - M$ to a node in M is the new inserted arc $\langle u,v\rangle$, but $\phi'(u) = \phi'(v) + 1$, and hence this can be drawn with no intersection. □

Theorem 7. *Let $G = (N,A)$ be a dag, with $n = |N|$ and $m = |A|$, and let T be a depth-first-search tree of G. Let us consider a sequence of k arcs insertion, and let $G' = (N,A')$ be the final acyclic graph, with $m' = |A'| = m + k$. Updating the DFS-tree after the insertion of each arc by using algorithm* Insert *requires $O(m' \cdot \min\{k,n\})$ worst-case total time for the whole sequence, and $O(n + m')$ for each arc insertion.*

Proof. Procedure P-DFS performs a partial DFS after each arc insertion, and its $O(n+m')$ time bound dominates the cost of the update. If a node w is marked by algorithm P-DFS due to the insertion of an arc $\langle u,v\rangle$ (with v possibly coincident with w) then a path from u to w has been introduced. Hence, from now on, $\phi(u) > \phi(w)$, and no further arc insertion can reverse the relative position of u and w: this means that, due to the pruning rules of the partial DFS performed by procedure P-DFS, node w will never be marked again due to the insertion of an arc $\langle u,v'\rangle$, for any v'. This lets us conclude that any node is marked at most n times in any sequence of arc insertions.

Since an arc $\langle w,y\rangle$ is scanned by DFS in procedure P-DFS only when node w is marked (otherwise the search is pruned at w), it turns out that any arc is scanned at most n times for any sequence of arc insertions. □

5. Conclusions

In this paper we deal with the problem of maintaining a depth-first-search tree in a *dag*. Our solution outperforms the $O(n+m)$ off-line algorithm any time one must handle a sequence of $k > n$ arc insertions in a *dag* with n nodes. In fact, any sequence of k arc insertions can be handled by performing the required updates in the DFS-tree in total time $O(m' \cdot \min\{k,n\})$, while the off-line algorithm requires $O(m' \cdot k)$ time. In particular, over a sequence of $\Theta(m)$ arc insertions our algorithm requires $O(n)$ amortized time per operation.

On the other side our data structures have size not larger than the one representing the resulting graph. Furthermore we observe that our algorithm is very simple to implement and it is never worse than the linear off-line algorithm. It is possible to show that the proposed algorithm, with trivial adjustments, also updates a DFS-tree in general digraphs: in this case we can not bound the time complexity better than $O(n + m)$ worst case, although there is some evidence of an improvement over the off-line approach, due to the fact that not all the graph is visited in case of one arc insertion.

Dynamic DFS, due to its theoretical and practical interest, deserves further attention: the problem for undirected and general directed graphs is still open, as well as to devise a fully dynamic solution, consisting in dealing with interleaved arc insertions and deletions.

References

[1] A.V. Aho, J.E. Hopcroft and J.D. Ullman, *The Design and Analysis of Computer Algorithms* (Addison-Wesley, Reading, MA, 1974).

[2] T.H. Cormen, C.E. Leiserson and R.L. Rivest, *Introduction to Algorithms* (MIT Press, Cambridge, MA and McGraw-Hill, New York, 1990).

[3] S.A. Cook, An observation on time-storage trade-off, *J. Comput. System Sci.* 9 (3) (1974) 308-316.

[4] P.G. Franciosa, G. Gambosi and U. Nanni, On the structure of DFS-forests on directed graphs and the dynamic maintenance of DFS on DAG's, in: *Proc. Ann. European Symp. on Algorithms* Lecture Notes in Computer Science, Vol. 855 (Springer, Berlin, 1994).

[5] J. Hopcroft and R.E. Tarjan, Efficient algorithms for graph manipulation, *Comm. ACM* 16 (1973) 372-378.

[6] N.D. Jones and W.T. Laaser, Complete problems for deterministic polynomial time, *Theoret. Comput. Sci.* 3 (1976) 105-117.

[7] R.E. Ladner, The circuit value problem is log-space complete for P, *SIGACT News* 7 (1) (1975) 18-20.

[8] U. Manber, *Introduction to Algorithms* (Addison-Wesley, Reading, MA, 1989).

[9] A. Marchetti-Spaccamela, U. Nanni and H. Rohnert, On-line graph algorithms for incremental compilation, in: *Proc. Workshop on Graph-Theoretic Concepts in Computer Science* (1993) 70-86.

[10] J.H. Reif, Depth-first search is inherently sequential, *Inform. Process. Lett.* 20 (1985) 229-234.

[11] J.H. Reif, A topological approach to dynamic graph connectivity, *Inform. Process. Lett.* 25 (1987) 65-70.

[12] R.E. Tarjan, Depth-first search and linear graph algorithms, *SIAM J. Comput.* 1 (1972) 146-160.